tier1_fleet_e2e

tier1_fleet_e2e.sh — Tier-1 MULTI-DEVICE FLEET OTA E2E on podman

Last verified: 2026-06-10 (run-id `20260610T111928Z-fleet`, fleet size 5, podman 5.8.2, arm64 Linux guest)

Overview

`tests/emulator/tier1_fleet_e2e.sh` proves **multi-device emulation** ("create emulators", plural): it boots **N distinct `ota-device-emu` containers concurrently** as a **fleet** against **one** containerized Helix OTA control plane, with no live hardware. The operator stages **one** real all-targets deployment at a version above the fleet's current; then every emulator (a distinct `hardware_id` → a distinct device) independently registers, gets a **200 update offer carrying the `deployment_id`**, applies the update, reports the **full telemetry lifecycle ACCEPTED** (`rejected=0`), and advances its version. The harness asserts **all N devices completed**, captures per-device evidence + a fleet summary, and cross-checks the **server fleet telemetry overview**.

Anti-bluff (§11.4 / §11.4.119): each emulator is its **own distinct device** — no two co-drive a shared exclusive resource; the fleet is genuinely concurrent (containers run in parallel). Every PASS is backed by real captured per-device Outcome JSON + the server `GET /telemetry/overview` body under `docs/qa/<run-id>-fleet/`. A device that does not complete is reported `[FAIL]` with its Outcome and the harness exits non-zero — it never fakes green.

Prerequisites

Identical to `tier1_full_lifecycle_e2e.sh`: a running podman machine (arm64 guest), a host Go toolchain, and `openssl`(≥3 ed25519)/`xxd`/`base64`/`curl`/`jq`/`python3`. The ed25519 trust keypair is ephemeral and never committed (§11.4.10).

Usage

```
bash tests/emulator/tier1_fleet_e2e.sh
        # default 5 devices
FLEET_SIZE=8 bash tests/emulator/tier1_fleet_e2e.sh    # N devices
KEEP_UP=1   bash tests/emulator/tier1_fleet_e2e.sh
        # leave the stack up to inspect
```

Optional environment overrides: `PODMAN`, `CP_IMAGE`, `EMU_IMAGE`, `ADMIN_USER`, `ADMIN_PASS`, `HELIX_PORT`, `FLEET_SIZE` (must be ≥ 2 to prove a fleet).

What it does (phases)

- **KEYS (0) / BUILD (a,b)** — same as the full-lifecycle harness: ephemeral ed25519 key, cross-compile static `linux/arm64` binaries, build the control-plane
  - `containers/`-submodule emulator base + derived emulator images.
- **BOOT + HEALTH (c)** — create a pod publishing `HELIX_PORT`, start the control plane with the trust key; **stale-listener guard** + gate on `/healthz` 200 AND `/api/v1/auth/login` answering.
- **STAGE (d)** — operator login → sign + upload artifact (201 `verified=true`) → create release → create **one** all-targets deployment (capture `deployment_id`).
- **FLEET (e)** — launch `FLEET_SIZE` detached emulator containers, each with a distinct `hardware_id`, all on `current-version` below the release. They run **concurrently**.
- **WAIT (f)** — poll each container to `exited` (bounded deadline).
- **ASSERT (g)** — per device: collect the container's stdout (its Outcome JSON) + exit code; assert `rc=0`, non-empty `device_id`, `applied=true`, `from`==current, `to`==release, `telemetry_rejected=0`, `telemetry_accepted≥1`, and `deployment_id`==staged. Count completed devices; emit a `fleet_summary.txt` table.
- **SERVER CROSS-CHECK (h)** — `GET /telemetry/overview`: assert the fleet-wide `event_counts.success` ≥ `FLEET_SIZE`.
- **VERDICT** — PASS only when **all N** devices completed **and** the server recorded ≥ N success events.

Outputs / evidence

Written under `docs/qa/<run-id>-fleet/`:

- `tier1_fleet_transcript.txt` — full timestamped run transcript.
- `fleet_summary.txt` — per-device table (container, hardware_id, rc, applied, from→to, telemetry accepted/rejected).
- `fleet/<container>_outcome.json` (+ `.log`) — each device's structured Outcome.
- `server_telemetry_overview.json` — the server fleet-wide event counts / by-state.
- `control_plane_boot.log` / `control_plane_after_fleet.log` — control-plane logs.

A representative passing overview: `total: 25` events for 5 devices (`download_started/installing/installed/verifying/success` × 5), `by_state: {success: 5}`, `failure_rate: 0`.

Edge cases & honest notes

- **One deployment, many devices.** A single all-targets deployment is resolved release-based for each device's model/os, so every fleet member is offered the same update and stamps the same `deployment_id` on its telemetry. As with the full-lifecycle harness, `target_count` at deploy time reflects the currently-registered set (0 here, since devices register afterward) — benign.
- **Concurrency is real.** The containers are launched detached and run in parallel; the harness only blocks in the WAIT phase to collect terminal state. Each device is independent (§11.4.119) — no shared exclusive resource is co-driven.
- **Stale listener / cleanup** — same guards as the full-lifecycle harness; `trap … EXIT INT TERM` removes the pod (all N+1 containers) and the ephemeral key + staging files unless `KEEP_UP`.

Related scripts

- `tests/emulator/tier1_full_lifecycle_e2e.sh` — the single-device full-lifecycle harness this one fans out into a fleet.
- `tests/emulator/tier1_container_e2e.sh` — the on-target-204 Tier-1 round-trip.
- `tests/e2e/pipeline_signed.sh` — the signed artifact-pipeline contract.